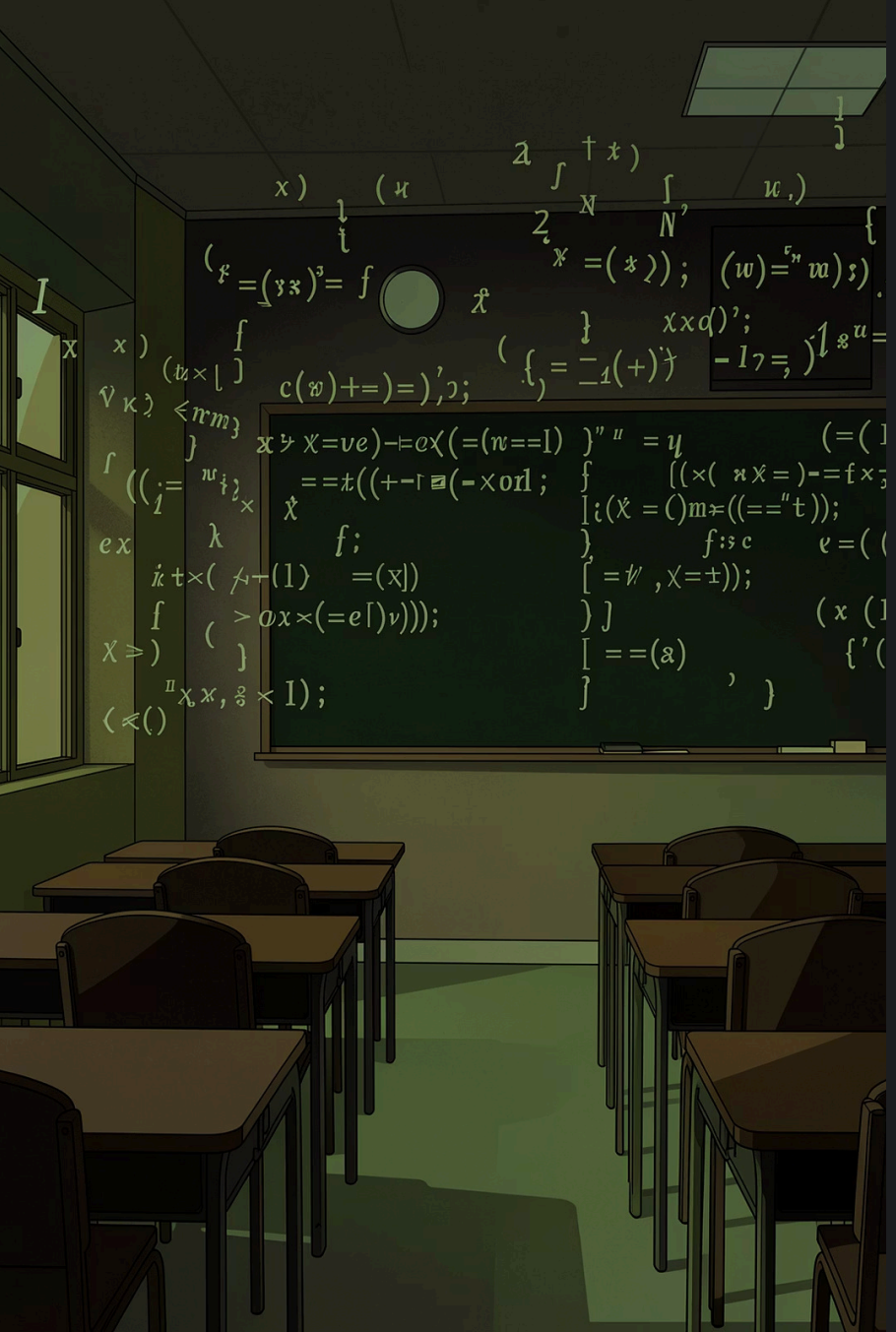


Объектно-ориентированно е программирование

Цели занятия: понять, что такое класс и объект; изучить базовый синтаксис класса в Python; научиться создавать объекты и писать простые методы.

Для кого

Ученики 9-го класса и преподаватели информатики

Фокус

Практический пример — класс Student

Результат

Уверенное создание классов и методов на Python

Made with GAMMA

Всё вокруг — объект

В ООП объект — это конкретная вещь с характеристиками (атрибутами) и поведением (методами). Класс — это чертёж, шаблон, по которому создаются объекты.

Атрибуты (state)	**Методы (behavior)**
name — имя ученика	add_score(score) — добавить оценку
grade — класс/курс	average() — вычислить средний балл
scores — список оценок	может быть метод print_info()

Итого: класс описывает общие параметры, объект хранит реальные значения.

Made with GAMMA

Код: класс Student

Ниже — полный пример класса с комментариями по self, конструктору и методам.

```
class Student:
    def __init__(self, name, grade):
        self.name = name          # имя (атрибут)
        self.grade = grade        # класс/курс (атрибут)
        self.scores = []          # список оценок
```

```
    def add_score(self, score):
        self.scores.append(score)
# self — ссылка на объект
    def average(self):
        return sum(self.scores)/len(self.scores) if self.scores else 0
```

Пояснение: self — это текущий экземпляр класса; __init__ — конструктор, создаёт и инициализирует объект.

Made with GAMMA

Создание объектов и взаимодействие

Создадим два объекта одного класса и вызовем методы:

```
s1 = Student("Анна", 9)
s2 = Student("Игорь", 9)

s1.add_score(5)
s1.add_score(4)

s2.add_score(3)
s2.add_score(4)

print(s1.average())  # средний Анны
print(s2.average())  # средний Игоря
```

Сравнение: оба — экземпляры Student, но имеют свои атрибуты (name, scores). Методы работают с конкретным объектом через self.

Made with GAMMA

Специальный метод __str__

__str__ определяет, как объект представляется в виде строки — удобно при печати и отладке.

```
def __str__(self):
    return f"{self.name}, grade {self.grade}"
```

Пример использования:

```
print(s1)  # выводит: Анна, grade 9
```

Зачем нужен: делает вывод понятным для людей, упрощает логирование и отображение объектов в интерфейсах.

Made with GAMMA

Задания трёх уровней

Создайте по одному классу для каждого задания. Минимум 2 атрибута и 2 метода.

Уровень 1 — BankAccount

Атрибуты: owner, balance. Методы: deposit(amount), withdraw(amount).

Уровень 2 — Book

Атрибуты: title, author, available (bool). Методы: borrow(), return_book().

Уровень 3 — Свободный выбор

Придумайте класс (например, Game, Timer, Playlist) с минимум 2 атрибутами и 2 методами.

Рекомендуется добавить проверки (например, отрицательный баланс) и docstring для каждого класса.

Made with GAMMA

Шпаргалка: шаблон класса и ключевые термины

Шаблон класса

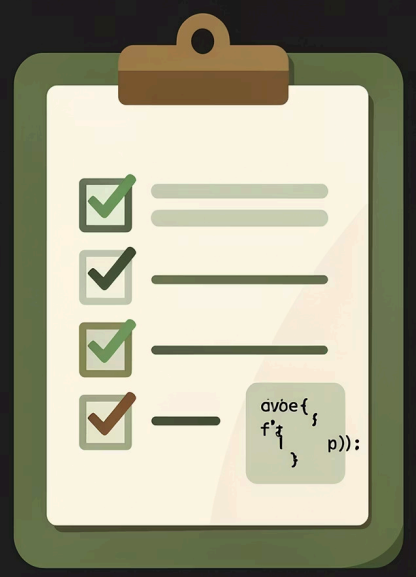
```
class Name:
    def __init__(self, ...):
        self.attr = value
    def method(self, ...):
```

Ключевые термины

- Класс — шаблон
- Объект (экземпляр) — конкретный объект
- Атрибуты — данные объекта
- Методы — функции, связанные с объектом
- self — ссылка на текущий экземпляр
- Инкапсуляция, Наследование, Полиморфизм — принципы ООП (общее знакомство)

Совет: комментируйте код и проводите тесты — создавайте небольшие примеры для проверки методов.

Made with GAMMA



Домашнее задание

1) В классе Student добавьте метод `is_passing(self, threshold=3)`, который возвращает True, если средний балл ≥ threshold.

2) Напишите код, который:

1. Создаёт список студентов с оценками
2. Фильтрует только успевающих (is_passing)
3. Выводит их имена и средний балл

```
# пример
students = [s1, s2, s3]
passing = [s for s in students if s.is_passing(3)]
for s in passing:
    print(s.name, s.average())
```

Критерии оценки

- Рабочий метод is_passing — 40%
- Корректная фильтрация и вывод — 40%
- Комменты и оформление кода — 20%

Made with GAMMA